Actividad 1: Revisión Integral (Python & C++)

Proyecto CALCULADORAEDAD

Ian Quiroga, Santino Laudati y Dante Iglesias

EEST N.º 1 “Eduardo Ader” - Vicente López

Laboratorio de Programación - 5.º Año 3.º A-B

Profesor: Mansilla Muñoz York Elías

Yamil De Jesus Ganduglia

2026

Fecha de entrega: 4 de noviembre de 2026

Repositorio de GitHub: https://github.com/pachusUwu/calculadoraedad.git

Control de versiones del documento

Versión	Fecha	Descripción	Autores
v1.0.0	04/11/2026	Baseline Aprobado	I. Quiroga, S. Laudati, D. Iglesias

Introducción

El presente informe documenta la resolución de la Actividad 1 de Laboratorio de Programación, cuyo objetivo fue repasar los conceptos de lógica y algoritmos trabajados el año anterior, implementando el mismo problema en dos lenguajes: Python, un lenguaje interpretado de alto nivel, y C++, un lenguaje compilado y estructurado de nivel más bajo.

El problema consiste en calcular la edad exacta de una persona a partir de su fecha de nacimiento, tomando como fecha actual el 8 de julio de 2026. Antes de calcular, el programa aplica un *filtro de consistencia* que verifica que la fecha ingresada exista realmente en el calendario: controla el rango del año, la cantidad de días de cada mes y el caso especial de febrero en los años bisiestos.

A lo largo del trabajo se describe la organización del proyecto, el código desarrollado en ambos lenguajes, las pruebas realizadas con sus capturas de pantalla y una comparación entre los dos entornos.

Desarrollo

Integrantes y tareas realizadas

Tabla 1

Distribución de tareas del grupo

Integrante	Tareas realizadas
Ian Quiroga	Programación de `proyecto_cpp/main.cpp` y `proyecto_python/main.py`, agregado de la validación de datos no numéricos y de fechas futuras, compilación con g++ (MSYS2) y ejecución en VS Code.
Santino Laudati	Redacción del informe APA v7, del manual del programador y del README; análisis comparativo entre Python y C++.
Dante Iglesias	Plan de pruebas (casos válidos e inválidos), capturas de pantalla, armado de la estructura de carpetas, `.gitignore`, `LICENSE` y repositorio Git (ramas `main` y `development`).

Estructura de carpetas del proyecto

Se respetó la estructura solicitada en la consigna:

```
calculadoraedad/
├── .gitignore            <- excluye ejecutables (.exe) y la carpeta .vscode/
├── README.md             <- descripción breve del ejercicio
├── LICENSE               <- licencia MIT de uso escolar
├── docs/
│   ├── InformeProyectoCalculadoraEdad.pdf
│   └── manuales/manual_programador_v1.0.0.(md|pdf)
├── capturas/             <- evidencias de ejecución (.png)
├── proyecto_cpp/main.cpp
└── proyecto_python/main.py
```

Algoritmo y filtro de consistencia

El programa se divide en dos funciones y un bloque principal. La función `esBisiesto(anio)` devuelve verdadero cuando el año es divisible por 4 y no por 100, o cuando es divisible por 400. La función `esFechaValida(d, m, a)` rechaza la fecha si el año está fuera del rango 1900–2026, si el mes no está entre 1 y 12, si el día no está entre 1 y 31, si el mes tiene 30 días (abril, junio, septiembre y noviembre) y se ingresó el día 31, o si en febrero se supera el día 28 (o el 29 en años bisiestos).

Para calcular la edad se resta el año de nacimiento al año actual y, si la persona todavía no cumplió años en 2026 (el mes actual es menor que el de nacimiento, o es el mismo mes pero el día todavía no llegó), se resta uno:

```
int edad = anioA - anioN;
if (mesA < mesN || (mesA == mesN && diaA < diaN))
{
    edad--;   // todavia no cumplio anios este anio
}
```

Además del esqueleto provisto, el grupo agregó dos controles: en C++ se verifica `cin.fail()` para detectar cuando el usuario escribe letras en lugar de números (en Python esto lo resuelve el bloque `try/except ValueError`), y en ambos lenguajes se rechaza una fecha posterior al 8/7/2026 dentro del mismo año, que de otro modo daría una edad negativa.

Comparación entre Python y C++

Tabla 2

Diferencias observadas entre ambas implementaciones

Aspecto	C++	Python
Ejecución	Se compila con `g++` y se genera un `.exe`	Lo interpreta `python` línea por línea
Tipos de datos	Declarados explícitamente (`int diaN`)	Dinámicos; se convierte con `int(input())`
Bloques	Llaves `{ }` y punto y coma	Indentación obligatoria
Operadores lógicos	`&&`, `||`, `!`	`and`, `or`, `not`
Entrada inválida	Se detecta con `cin.fail()`	Se captura la excepción `ValueError`
Lectura	Una sola línea con tres valores	Tres `input()` separados

La lógica del algoritmo es idéntica en los dos lenguajes; lo que cambia es la sintaxis y la forma de manejar errores. Python resulta más rápido de escribir, mientras que C++ obliga a pensar en los tipos de datos y en el paso de compilación.

Pruebas realizadas

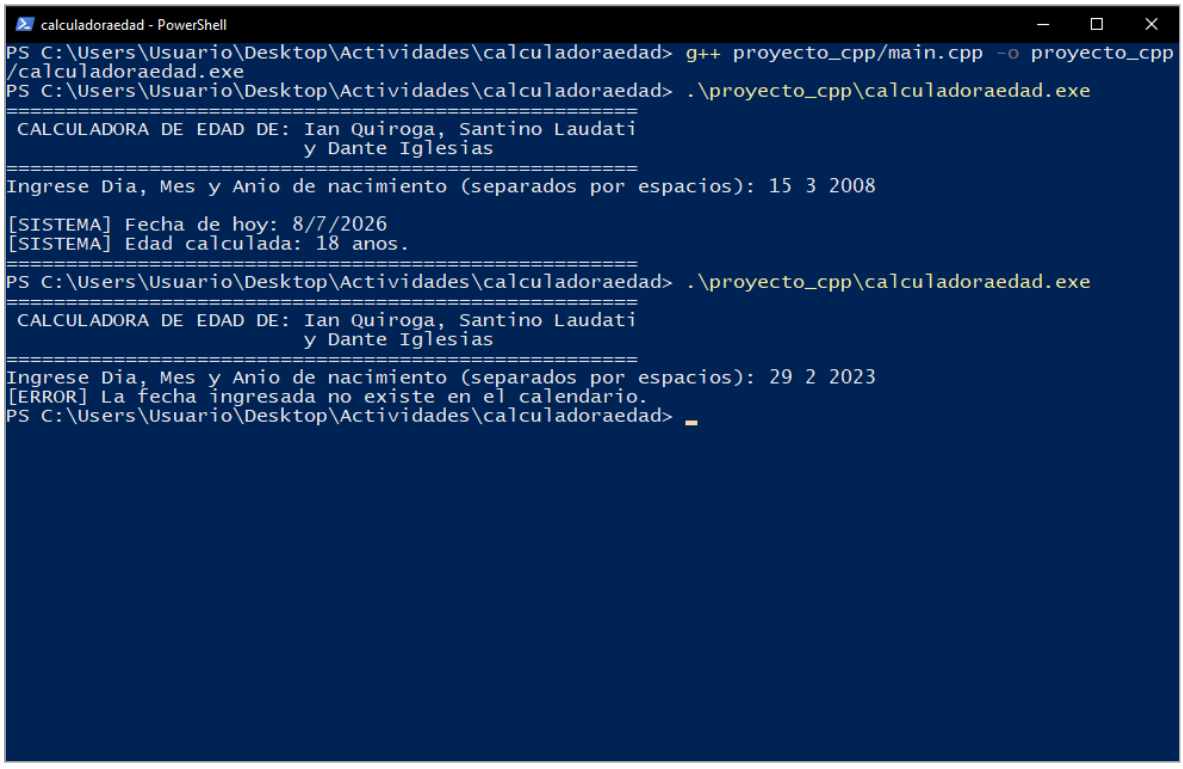
Tabla 3

Casos de prueba ejecutados (fecha actual 8/7/2026)

Entrada (d m a)	Resultado esperado	Resultado obtenido
15 3 2008	18 años	18 años ✓
8 7 2008 (cumple hoy)	18 años	18 años ✓
9 7 2008 (cumple mañana)	17 años	17 años ✓
29 2 2024 (bisiesto)	2 años	2 años ✓
29 2 2023 (no bisiesto)	Error de fecha	Error de fecha ✓
31 4 2000 (abril tiene 30)	Error de fecha	Error de fecha ✓
1 1 2027 (fuera de rango)	Error de fecha	Error de fecha ✓
abc	Error de dato no numérico	Error de dato no numérico ✓

Figura 1

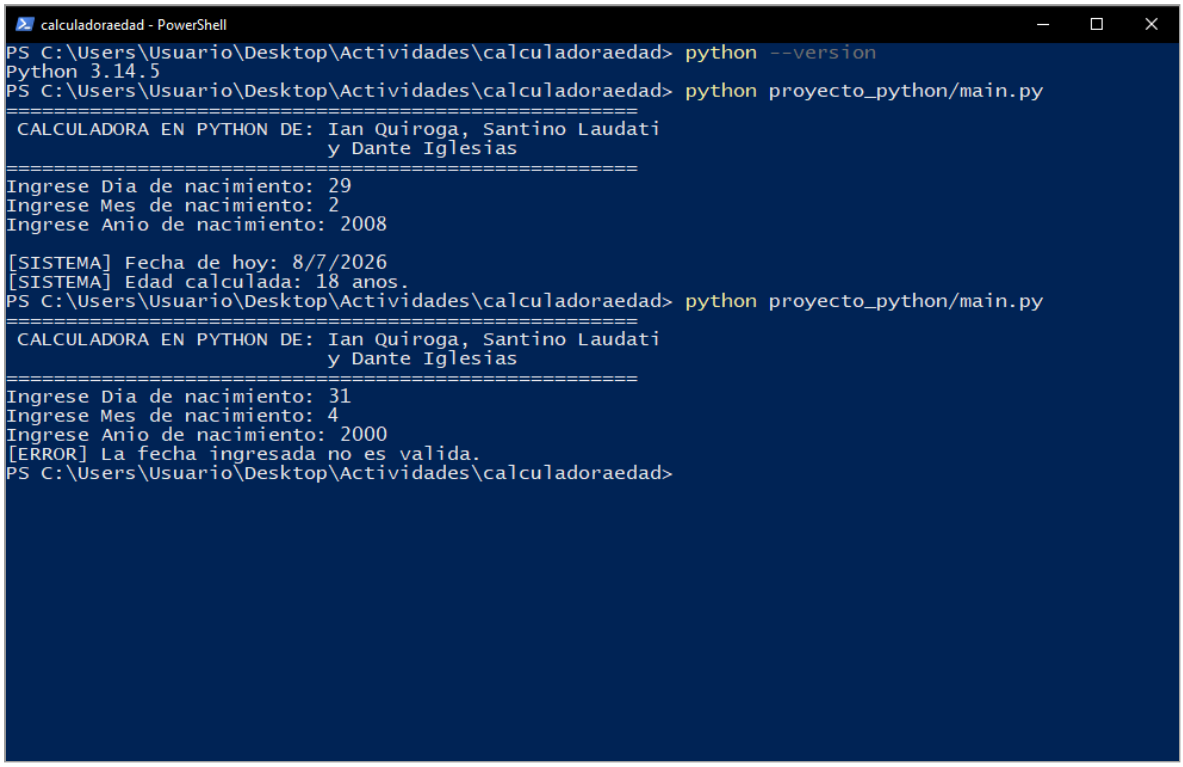
Compilación con g++ y ejecución del programa en C++ (fecha válida y 29/2/2023 inválida)



Nota. Captura tomada en la terminal de PowerShell con el compilador g++ 16.1.0 de MSYS2 UCRT64.

Figura 2

Ejecución del programa en Python con una fecha válida y una fecha inexistente (31 de abril)



Compilación y ejecución

Desde la raíz de la carpeta `calculadoraedad` en la terminal de VS Code se utilizaron los siguientes comandos:

```
g++ proyecto_cpp/main.cpp -o proyecto_cpp/calculadoraedad.exe
.\proyecto_cpp\calculadoraedad.exe
python proyecto_python/main.py
```

Repositorio

El proyecto se versionó con Git en la rama `main`, con una rama `development` donde se integraron los cambios antes de unirlos a `main`. El ejecutable no se sube al repositorio porque está excluido en el `.gitignore`. Las direcciones del repositorio figuran en las Referencias.

Conclusión

La actividad permitió retomar la lógica de programación del año anterior y comprobar que un mismo algoritmo puede escribirse en dos lenguajes muy distintos sin cambiar su razonamiento. El filtro de consistencia demostró la importancia de no confiar en los datos que ingresa el usuario: sin esas condiciones el programa aceptaría fechas imposibles como el 31 de abril o el 29 de febrero de un año no bisiesto.

Como grupo nos resultó más cómodo Python para escribir rápido, pero C++ nos obligó a entender mejor los tipos de datos y el proceso de compilación. También aprendimos que las pruebas tienen que incluir casos límite (el día exacto del cumpleaños, el día siguiente, un año bisiesto) porque ahí es donde aparecen los errores de lógica.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

cplusplus.com. (s.f.). *Control structures*. Recuperado el 4 de noviembre de 2026, de https://cplusplus.com/doc/tutorial/control/

Mansilla Muñoz, Y. E. (2026). *Actividad 1: De revisión integral (Python & C++)* [Material de clase]. E.E.S.T. N.° 1 "Eduardo Ader".

MSYS2. (s.f.). *MSYS2: Software distribution and building platform for Windows*. https://www.msys2.org/

Python Software Foundation. (s.f.). *El tutorial de Python*. https://docs.python.org/es/3/tutorial/

Quiroga, I., Laudati, S., & Iglesias, D. (2026). *calculadoraedad* [Repositorio de código fuente]. GitHub. https://github.com/pachusUwu/calculadoraedad